HITEC University Taxila

Department of Cyber Security

Course: Essentials of Artificial Intelligence

Lab Manual 6

Submitted By : Muhammad Sannan Azmat

Reg No: 23-CyS-010

Submitted To: Mubashir Iqbal

DOS: 03/27/2026

Lab Task

Task 1. Do these tasks:

a) Modify the environment to add a third room.

b) Make the agent move randomly when both rooms are clean.

c) Add a counter to track how many times the agent cleaned.

Input

lab6 task 1.py > ...

```python
import random
environment = {
    "Room1": "Dirty",
    "Room2": "Dirty",
    "Room3": "Dirty"
}
agent_location = "Room1"
clean_counter = 0
def move_agent():
    global agent_location
    agent_location = random.choice(list(environment.keys()))
    print(f"Agent moved to {agent_location}")
def clean_room():
    global clean_counter
    if environment[agent_location] == "Dirty":
        environment[agent_location] = "Clean"
        clean_counter += 1
        print(f"Agent cleaned {agent_location}")
    else:
        print(f"{agent_location} is already clean")
for step in range(10):
    print(f"\nStep {step+1}: Agent in {agent_location}")
    clean_room()
    if all(state == "Clean" for state in environment.values()):
        print("All rooms are clean! Agent will move randomly.")
        move_agent()
    else:
        rooms = list(environment.keys())
        current_index = rooms.index(agent_location)
        agent_location = rooms[(current_index + 1) % len(rooms)]
print("\nFinal Environment State:", environment)
print("Total times agent cleaned:", clean_counter)
```

Output

```
task 1.py"

Step 1: Agent in Room1
Agent cleaned Room1

Step 2: Agent in Room2
Agent cleaned Room2

Step 3: Agent in Room3
Agent cleaned Room3
All rooms are clean! Agent will move randomly.
Agent moved to Room3

Step 4: Agent in Room3
Room3 is already clean
All rooms are clean! Agent will move randomly.
Agent moved to Room3

Step 5: Agent in Room3
Room3 is already clean
All rooms are clean! Agent will move randomly.
Agent moved to Room2

Step 6: Agent in Room2
Room2 is already clean
All rooms are clean! Agent will move randomly.
Agent moved to Room2
```

```
Step 7: Agent in Room2
Room2 is already clean
All rooms are clean! Agent will move randomly.
Agent moved to Room3

Step 8: Agent in Room3
Room3 is already clean
All rooms are clean! Agent will move randomly.
Agent moved to Room3

Step 9: Agent in Room3
Room3 is already clean
All rooms are clean! Agent will move randomly.

Step 9: Agent in Room3
Room3 is already clean
All rooms are clean! Agent will move randomly.
Step 9: Agent in Room3
Room3 is already clean
All rooms are clean! Agent will move randomly.
All rooms are clean! Agent will move randomly.
Agent moved to Room1

Agent moved to Room1


Step 10: Agent in Room1
Room1 is already clean
Step 10: Agent in Room1
```

```
Room1 is already clean
Room1 is already clean
All rooms are clean! Agent will move randomly.
All rooms are clean! Agent will move randomly.
Agent moved to Room3

Agent moved to Room3


Final Environment State: {'Room1': 'Clean', 'Room2': 'Clean', 'Room3': 'Clean'}
Final Environment State: {'Room1': 'Clean', 'Room2': 'Clean', 'Room3': 'Clean'}
Total times agent cleaned: 3
PS S:\AI Lab>
```

Task 2. Design a rule-based reflex agent for a smart light system.

Input

lab6 task 2.py > ...

```python
class SmartLightAgent:
    def __init__(self):
        self.light_on = False
    def perceive_environment(self, motion_detected, ambient_light_level):
        """
        motion_detected: True if someone is in the room
        ambient_light_level: value from 0 (dark) to 10 (very bright)
        """
        return {"motion": motion_detected, "light": ambient_light_level}
    def rule_based_action(self, percepts):
        if percepts["motion"] and percepts["light"] < 5:
            self.light_on = True
            return "Light turned ON (motion + dark)."
        elif not percepts["motion"]:
            self.light_on = False
            return "Light turned OFF (no motion)."
        elif percepts["motion"] and percepts["light"] >= 5:
            self.light_on = False
            return "Light kept OFF (room already bright)."
        return "No action taken."
agent = SmartLightAgent()
scenarios = [
    (True, 2),
```

```python
    (True, 8),
    (False, 3),
    (False, 9)
]
for i, (motion, light) in enumerate(scenarios, 1):
    percepts = agent.perceive_environment(motion, light)
    action = agent.rule_based_action(percepts)
    print(f"Scenario {i}: Motion={motion}, Light={light} → {action}")
```

Output

```
PS S:\AI Lab> & C:/Users/HP/AppData/Local/Programs/Python/Python313/pytho
task 2.py"
Scenario 1: Motion=True, Light=2 → Light turned ON (motion + dark).
Scenario 2: Motion=True, Light=8 → Light kept OFF (room already bright).
Scenario 3: Motion=False, Light=3 → Light turned OFF (no motion).
Scenario 4: Motion=False, Light=9 → Light turned OFF (no motion).
PS S:\AI Lab>
```

Task 3. Simulate a thermostat agent that maintains room temperature between 20°C and 24°C using internal memory (model of past temperatures):

a) An agent can sense the current temperature.

b) It must remember the past few temperature values (model) to predict if heating/cooling is needed.

c) If the temperature trend is rising or falling rapidly, take preventive actions.

Input

lab6 task 3.py > ThermostatAgent

```python
import random
class ThermostatAgent:
    def __init__(self):
        self.memory = []
        self.heating = False
        self.cooling = False
    def sense_temperature(self):
        return random.randint(18, 26)
    def update_memory(self, temp):
        self.memory.append(temp)
        if len(self.memory) > 5:
            self.memory.pop(0)
    def analyze_trend(self):
        if len(self.memory) < 2:
            return "stable"
        diff = self.memory[-1] - self.memory[-2]
        if diff > 1.5:
            return "rising rapidly"
        elif diff < -1.5:
            return "falling rapidly"
        else:
            return "stable"
    def decide_action(self, temp):
        trend = self.analyze_trend()
        if temp < 20 or (trend == "falling rapidly" and temp <= 21):
```

```python
            self.heating = True
            self.cooling = False
            return f"Heating ON (temp={temp}°C, trend={trend})"
        elif temp > 24 or (trend == "rising rapidly" and temp >= 23):
            self.cooling = True
            self.heating = False
            return f"Cooling ON (temp={temp}°C, trend={trend})"
        else:
            self.heating = False
            self.cooling = False
            return f"No action (temp={temp}°C, trend={trend})"
agent = ThermostatAgent()
for step in range(10):
    temp = agent.sense_temperature()
    agent.update_memory(temp)
    action = agent.decide_action(temp)
    print(f"Step {step+1}: Current Temp={temp}°C → {action}")
```

Output

```
PS S:\AI Lab> & C:/Users/HP/AppData/Local/Programs/Python/Python313/python.e
task 3.py"
Step 1: Current Temp=19°C → Heating ON (temp=19°C, trend=stable)
Step 2: Current Temp=24°C → Cooling ON (temp=24°C, trend=rising rapidly)
Step 3: Current Temp=22°C → No action (temp=22°C, trend=falling rapidly)
Step 4: Current Temp=25°C → Cooling ON (temp=25°C, trend=rising rapidly)
Step 5: Current Temp=23°C → No action (temp=23°C, trend=falling rapidly)
Step 6: Current Temp=18°C → Heating ON (temp=18°C, trend=falling rapidly)
Step 5: Current Temp=23°C → No action (temp=23°C, trend=falling rapidly)
Step 6: Current Temp=18°C → Heating ON (temp=18°C, trend=falling rapidly)
Step 6: Current Temp=18°C → Heating ON (temp=18°C, trend=falling rapidly)
Step 7: Current Temp=25°C → Cooling ON (temp=25°C, trend=rising rapidly)
Step 7: Current Temp=25°C → Cooling ON (temp=25°C, trend=rising rapidly)
Step 8: Current Temp=23°C → No action (temp=23°C, trend=falling rapidly)
Step 9: Current Temp=24°C → No action (temp=24°C, trend=stable)
Step 10: Current Temp=20°C → Heating ON (temp=20°C, trend=falling rapidly)
PS S:\AI Lab>
```

Conclusion

In this lab, we extended the vacuum cleaner agent by adding a third room, random movement when all rooms are clean, and a cleaning counter to track performance. We then designed a rule-based reflex agent for a smart light system that reacts to motion and ambient brightness. Finally, we simulated a thermostat agent that uses internal memory to monitor past

temperatures, detect trends, and take preventive heating or cooling actions. Collectively, these tasks demonstrate how agents can combine simple rules with memory to make intelligent, proactive decisions in dynamic environments.